- **Background Processes:** These are non-interactive or daemon processes. They are self-executable and do not need any user input and interaction.

When the operating system gets started, the kernel initiates a few processes on its own that are called init processes. This runs several shell scripts of /etc. directory. These init scripts, in turn, can start several daemon services or programs that are just background programs without any user interface.

In Linux OS, a process can launch several other processes that are called parent and child processes. Daemon processes are background processes.

Process Attributes

Attributes or properties of Processes are listed below:

i) PID: PID means ID of the process. Every process of Linux/Unix OS has a unique identification number that is called process-id. Kernel of the Unix OS uses this identification number to identify the process.

ii) PPID: Every process in Unix, is created by some other methods. The process that creates another process is known as the parent process and the created process is known as a child process. PPID is the ID of the parent process.

iii) TTY: TTY is the terminal to which the process is associated. Unix commands are run by the terminal that is associated with a process. All processes are not associated with the terminal. The processes that do not belong to any terminal are known as daemon processes.

iv) UID: UID is the user Id of the owner of the process.The user or owner of the process that is a root user can kill any process. Users can also set the file access permission as well for the processes.

v) File Descriptors: Mainly, three file descriptors are related to the processes, input, and output and error descriptors.

Process States

During execution, a Unix process has to go or shift from one state to another. Depending on the environment, the processes may change their states. UNIX processes can be in any of the below-listed states after its creation:

i) Running: If the process is executed by the processor, then it will be called its running state. Moreover, if any process is ready to run (waiting for some I/O or processor allocation), then it will also be in running state.

ii) Waiting: A process may have to wait for any system resource, user I/O or any event. The UNIX kernel also divides this waiting state to interruptible and non-interruptible states.